Contents

app_2 – il bootloader caOS: guida utente **1**
1. Mappa della flash . . . 1
2. Le immagini . . . 1
   2.1 Formato `.cimg` . . . 1
   2.2 Costruire un'immagine . . . 2
   2.3 Chiavi . . . 2
3. Avvio . . . 2
   3.1 Il canale fra bootloader e immagine . . . 2
4. Aggiornare . . . 3
   4.1 Dal recovery . . . 3
   4.2 Dal recovery, via USB (DFU) . . . 3
   4.3 Dall'immagine in esecuzione . . . 3
5. Il lato immagine (`EN_BOOT_IMAGE`) . . . 4
6. CLI del bootloader . . . 4
7. Prova completa . . . 4
8. Limiti noti . . . 5

app_2 – il bootloader caOS: guida utente

`app_2` è il bootloader di caOS per **NUCLEO-H563ZI** e **NUCLEO-H753ZI**. È una normale applicazione caOS: usa driver, rete, flash, SHA-256 ed Ed25519 delle librerie, quindi chiunque abbia il pacchetto può costruirne uno proprio.

Cosa fa:

- all'avvio sceglie uno slot, ne verifica l'immagine (SHA-256 e firma) e le passa la CPU con `ca_os_end()`, che spegne caOS e rimette l'hardware come dopo un reset;
- riceve le immagini nuove via rete (TFTP: il PC le spinge), anche dall'immagine in esecuzione, oppure via USB (DFU, `dfu-util`) quando è in recovery;
- avvia un'immagine nuova **in prova**, con l'IWDG armato; se l'immagine non si conferma entro tre avvii, torna a quella precedente (**rollback**).

---

1. Mappa della flash

Uguale sulle due board (2 MB in due banchi da 1 MB):

Area	Indirizzo	Dimensione	Note
bootloader (app_2)	`0x08000000`	256 KB	banco 1, non si aggiorna da sé; protezione da scrittura con `boot protect on`
slot A	`0x08040000`	768 KB	banco 1
slot B	`0x08100000`	768 KB	banco 2
conf di avvio	ultimo settore del banco 2	8 KB (H563) / 128 KB (H753)	la scrive solo il bootloader

La mappa è condivisa con le immagini in `caos/inc/boot_image.h` (`CA_BOOT_SLOT_A_BASE`, ...): una sola sorgente per entrambi i lati.

2. Le immagini

2.1 Formato `.cimg`

Uno slot contiene un header da 1 KB seguito dall'immagine, linkata per partire subito dopo (la tabella dei vettori resta allineata come VTOR pretende):

Byte	Campo	Contenuto
0	magic	`'CAIM'`
4	u16	versione dell'header (1)
6	u16	flags: b0 = firmata
8	u32	dimensione dell'immagine
12	u32	indirizzo della tabella dei vettori = base dello slot + 0x400
16	u32	versione
20	char[32]	tag, testo libero
52	u32	build time (epoch)
56	u8[32]	SHA-256 dei byte 0..55 + l'immagine
88	u8[64]	firma Ed25519 dei byte 0..87 (lo SHA dentro copre l'immagine)

2.2 Costruire un'immagine

Un'applicazione del workspace diventa immagine aggiungendo `image.cmake`:

```
set(CAOS_IMAGE_SLOTS A B)
```

La build produce `<mcu>_<app>_slotA.elf` e `_slotB.elf`, linkati ciascuno nel proprio slot. Poi:

```
./build_caos.sh H563 -wp
python3 tools/caos_image.py elfs/H563/h563_app_4_slotB.elf --version 3 --tag "v1.3" --sign
python3 tools/caos_image.py --info elfs/H563/h563_app_4_slotB.cimg
```

Un'immagine linkata per A non parte da B: il bootloader controlla l'indirizzo nell'header.

2.3 Chiavi

Il repository contiene una coppia di chiavi **demo**: `workspace/app_2/boot_ed25519.pem` (privata, pubblica di proposito) e `boot_pubkey.h` (la pubblica, compilata nel bootloader). Con quella chiunque può firmare immagini per l'esempio. Un prodotto vero si fa la sua coppia:

```
python3 tools/caos_image.py --keygen --key ~/secret/prod.pem   # rigenera boot_pubkey.h
python3 tools/caos_image.py <slot.elf> --sign ~/secret/prod.pem
```

Con `BOOT_REQUIRE_SIGNED 1` (in `boot_slot.h`, il default) partono solo immagini firmate con quella chiave.

3. Avvio

All'avvio, prima che esista qualunque task, il bootloader:

1. **conferma** – se lo slot in prova ha segnato la conferma, diventa lo slot attivo;
2. **richiesta di aggiornamento** – se l'immagine ha chiesto di provare l'altro slot, lo verifica e lo mette `pending`;
3. **prova** – se c'è uno slot `pending`: conta un tentativo; oltre `CA_BOOT_MAX_TRIES`
   (3) esegue il **rollback**: cancella lo slot e torna all'attivo. Altrimenti arma l'IWDG (`CA_BOOT_IWDG_MS`, 8 s) e avvia lo slot in prova;
4. **avvio normale** – lo slot attivo se valido, altrimenti il primo valido;
5. **recovery** – nessuna immagine valida, oppure B1 premuto al reset: il bootloader resta acceso con la console, la CLI e la rete.

Lo slot cancellato al rollback non è un dettaglio: senza, la regola "primo valido" potrebbe farlo ripartire senza IWDG il giorno in cui l'altro slot si guasta.

3.1 Il canale fra bootloader e immagine

Bootloader e immagine non condividono memoria: l'unico canale sono due **registri di backup** (`bkp.h`), che sopravvivono al reset fra i due.

Registro	Chi scrive	Contenuto
`CA_BOOT_BKP_STATE` (16)	bootloader; l'immagine alla conferma	magic, slot in prova, tentativi (`0xFF` = confermato)
`CA_BOOT_BKP_MSG` (17)	immagine	richiesta di aggiornamento per uno slot

La conferma sta nel registro di stato e non in quello dei messaggi: un'immagine confermata che poi chiede un aggiornamento non deve perdere la conferma.

! Sulle Nucleo **VBAT è collegato a VDD**: togliere l'alimentazione azzera i registri di backup. Un'immagine in prova riparte allora da zero tentativi, e una confermata ma non ancora registrata viene riprovata (e si riconferma). Nessuno dei due casi è pericoloso; con VBAT alimentato a batteria il problema non c'è.

4. Aggiornare

4.1 Dal recovery

Il bootloader in recovery prende un indirizzo con DHCP e lo stampa:

```
BOOT: update - tools/caos_tftp put 192.168.10.93 <image.cimg>
```

Dal PC (libreria standard, niente sudo: la porta 69 è quella della scheda):

```
python3 tools/caos_tftp put 192.168.10.93 elfs/H563/h563_app_4_slotA.cimg
```

Il bootloader accetta solo nomi `.cimg`, scrive nello slot che **non** partirebbe al reset, verifica l'immagine, la mette `pending` e si riavvia nella prova:

```
BOOT: update "h563_app_4_slotA.cimg" from 192.168.10.1 -> slot A, erasing ...
BOOT: update OK, slot A v1 "app_4 A", signed - pending, reset into its trial
BOOT: trial of slot A, start 1 of 3, IWDG 8000 ms
```

Un'immagine alterata o linkata per l'altro slot viene ricevuta ma dichiarata non valida, e non parte.

4.2 Dal recovery, via USB (DFU)

Il recovery espone anche un'interfaccia **DFU in modalità download** (`ca01:0040`, "caOS bootloader"), sulla porta USB usata da example_57 (H563: USB_DRD; H753: OTG_FS). Con `dfu-util` non serve la rete:

```
dfu-util -l                                    # Found DFU: [ca01:0040] ... name="caOS
    bootloader"
dfu-util -d ca01:0040 -a 0 -D elfs/H753/h753_app_4_slotA.cimg
```

Lo slot scelto, la verifica e la prova sono gli stessi del TFTP. Un'immagine non valida finisce con `dfuERROR , status(7) = Programmed memory failed verification` lato PC. Blocchi da 64 byte, uno per pacchetto di EP0: circa 12 s per 117 KB su H563, 45 s per 120 KB su H753 (settori da 128 KB), cancellazione compresa. Un aggiornamento alla volta: TFTP e DFU si escludono.

4.3 Dall'immagine in esecuzione

Un'immagine con `EN_BOOT_IMAGE` riceve la nuova immagine nell'**altro** slot e chiede al bootloader di provarla (`ca_boot_receive_update()`); vedi par.5. Il comando dal PC è lo stesso. Un'immagine ancora in prova e non confermata rifiuta l'aggiornamento: l'altro slot è la sua via di ritorno.

5. Il lato immagine (EN_BOOT_IMAGE)

caos/inc/boot_image.h, flag EN_BOOT_IMAGE (richiede EN_WDT):

```
char ca_boot_image_slot(void);       /* 'A', 'B', 0 = not started from a slot */
bool ca_boot_is_trial(void);         /* started as a trial, not confirmed yet */
void ca_boot_image_start(void);      /* in ca_os_main: on a trial, take the IWDG over */
void ca_boot_confirm(void);          /* healthy: keep this slot */
bool ca_boot_request_update(char slot);              /* resets */
bool ca_boot_receive_update(ca_mac_t mac, u32 ms);   /* TFTP into the other slot, resets */
```

ca_boot_image_start() riconfigura l'IWDG lasciato acceso dal bootloader e lo alimenta da un job: per CA_BOOT_TRIAL_MS (60 s) se l'immagine non si conferma, per sempre dopo ca_boot_confirm(). Un'immagine che non chiama ca_boot_image_start() non alimenta l'IWDG e viene resettata dopo 8 s: è il caso del rollback.

Quando confermare lo decide l'applicazione, ed è la scelta più importante: deve significare "funziono davvero". workspace/app_4, l'immagine dimostrativa, si conferma quando la rete è su (lease DHCP ottenuto).

```
void ca_os_main(void)
{
    ca_boot_image_start();
    /* ... jobs ... */
    ca_os_run(true);
}

/* in a task, once the application is healthy */
ca_boot_confirm();
```

6. CLI del bootloader

```
boot info          slots, conf (attivo, pending), motivo del recovery
boot select A|B    slot attivo al prossimo reset (senza prova)
boot go [A|B]      avvia subito uno slot
boot erase A|B     cancella uno slot
boot protect [on|off]  protezione da scrittura dell'area del bootloader (option byte)
tftp recv [sec]    riceve un file e lo scarta (prova della rete)
```

boot protect on scrive gli option byte (H563: gruppi WRP 0-7 di FLASH_WRP1R; H753: settori 0-1 di FLASH_WPSN1): la protezione resta dopo reset e spegnimento, e il banner la mostra. **Blocca anche la programmazione via SWD** finché non si fa boot protect off. Il sintomo cambia con la famiglia: sull'H753 prog.sh fallisce in cancellazione (failed to erase memory; dopo può servire reset_mcu.sh), sull'H563 la cancellazione salta in silenzio i settori protetti e l'errore arriva in verifica (Data mismatch). Sull'H563 quindi riscrivere *lo stesso* bootloader "riesce": i byte sono già quelli. In alternativa, da STM32CubeProgrammer si azzerano gli option byte di protezione.

app_4 aggiunge img info (slot, prova, registro di stato) e img confirm.

7. Prova completa

workspace/app_2/test_boot_pc.py esegue l'intero ciclo su una board e chiude con un verdetto:

```
./build_caos.sh H563 -wp
sudo -n ./tools/bench_net.sh up            # rete del banco: SOLO a casa
python3 workspace/app_2/test_boot_pc.py --mcu H563 --tty /dev/serial/by-id/...-if02
```

Fase	Cosa verifica
1	recovery con slot vuoti e indirizzo DHCP

Fase	Cosa verifica
2	immagine alterata non valida (SHA-256), nome non `.cimg` rifiutato
3	app_4 in A: prova con IWDG, conferma
4	aggiornamento dall'immagine in esecuzione: A registrato attivo, B in prova e confermato
5	immagine che non si conferma (app_1): tre prove, rollback, A cancellato, riparte B

Esito il 21/10/2026: **5/5 su NUCLEO-H563ZI e su NUCLEO-H753ZI**. Provati a mano su entrambe: DFU (immagine buona -> prova e conferma; alterata -> `errVERIFY`) e `boot protect` (on -> SWD bloccato, bootloader intatto; off -> SWD di nuovo possibile). Velocità DFU: ~10 KB/s su H563, ~2,7 KB/s su H753.

8. Limiti noti

- Il bootloader non si aggiorna da sé: un difetto nel bootloader si corregge dal SWD (con `boot protect off`, se la protezione è attiva).
- Nessuna protezione contro il downgrade: una versione più vecchia, firmata, viene accettata (scelta voluta).
- Chiunque sulla rete può spingere un file: è la firma che impedisce di avviarlo.